08 · Signals (signal, computed, effect)

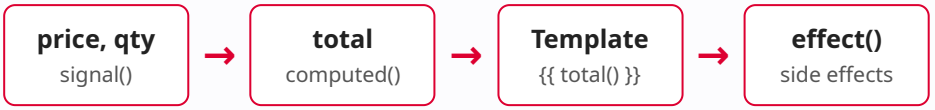
Beginner Angular learning notes

a. Definition

A **signal** is a box that holds a value and tells Angular whenever the value changes. You **read** it by calling it like a function (`count()`) and **write** with `.set()` or `.update()`. Because Angular knows exactly which templates read which signals, only the parts that need to change are refreshed.

`computed()` creates a read-only signal **derived** from other signals (it recalculates automatically and caches the result). `effect()` runs code (a side effect, like logging or saving) whenever the signals it reads change.

b. Diagram



Change flows automatically from source signals to derived values and the screen

c. Syntax

```
const count = signal(0);          // create
count();                          // read -> 0
count.set(5);                     // replace
count.update(c => c + 1);         // change based on old value
const double = computed(() => count() * 2);
effect(() => console.log(count()));
```

d. Example Program

Programs/08_Signals/cart-signals.component.ts

```
import { Component, computed, effect, signal, untracked } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <h3>Idli Order</h3>
    <button id="dec" (click)="dec()">-</button>
    <strong>{{ plates() }}</strong>
    <button id="inc" (click)="inc()">+</button>

    <p>Price per plate: ₹{{ price }}</p>
    <p>Total: ₹{{ total() }}</p>

    <h4>Change history (written by an effect)</h4>
    <ul>
      @for (h of history(); track $index) { <li>{{ h }}</li> }
    </ul>
  `,
})
export class CartSignalsComponent {
```

```
  price = 30;
  plates = signal(1); // writable signal

  // computed: derived, read-only, cached
  total = computed(() => this.plates() * this.price);

  history = signal<string[]>([]);

  constructor() {
    // effect: runs once now and again whenever plates() changes
    effect(() => {
      const message = `plates = ${this.plates()}`;
      // untracked: do not re-run this effect when history changes
      untracked(() => this.history.update((h) => [...h, message]));
    });
  }

  inc() { this.plates.update((n) => n + 1); }
  dec() { this.plates.update((n) => Math.max(1, n - 1)); }
}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Click **+** twice.
2. Click **-** once.

```
Idli Order
-2+
Price per plate: ₹30
Total: ₹60
Change history (written by an effect)
plates = 1
plates = 2
plates = 3
plates = 2
```

f. Real-World Example (English + தமிழ்)

English

At an idli shop, when you change the number of plates the bill total updates by itself, because the formula 'plates x price' is already known. A signal is the number of plates and `computed` is the bill.

தமிழ்

இட்லிக் கடையில் நீங்கள் ஆர்டர் செய்யும் தட்டுகளின் எண்ணிக்கையை (plates) மாற்றினால், பில் தொகை (total) தானாகவே மாறுகிறது. கடைக்காரர் ஒவ்வொரு முறையும் புதிதாகக் கணக்குப் போட வேண்டியதில்லை - ஏனெனில் 'தட்டு × விலை' என்ற சூத்திரம் அவர் மனதில் இருக்கிறது.

`signal` = தட்டுகளின் எண்ணிக்கை. `computed` = பில் தொகை (தானாக மாறும்). `effect` = ஒவ்வொரு ஆர்டர் மாற்றத்தையும் நோட்டுப் புத்தகத்தில் எழுதி வைப்பது போன்ற

கூடுதல் வேலை.

g. Pros

- Fine-grained updates (fast)
- Simple mental model
- No subscriptions to manage
- Works without Zone.js

h. Cons

- Different from RxJS, so two models to learn
- Effects can be overused

i. Advanced / New Features

- Signals stable (v17-v20)
- `linkedSignal` and `resource` (v19)
- Signal-based inputs, outputs and queries
- Experimental Signal Forms (v21)

j. Common Mistakes

- Forgetting the parentheses when reading a signal (`count` vs `count()`).
- Mutating an array/object inside a signal (`list().push(x)`); create a new one with `update` instead.
- Setting signals inside `computed()`; computed must be pure.
- Overusing `effect()` to copy state between signals; use `computed()` instead.

k. Interview Questions

Q1. What is a signal?
A reactive value container; reading it in a template or computed registers a dependency so Angular updates only what changed.

Q2. `computed` vs `effect`?
`computed` derives a value and returns a signal; `effect` performs a side effect and returns nothing.

Q3. `set` vs `update`?
`set` assigns a new value; `update` receives the old value and returns the new one.

Q4. How are signals different from RxJS Observables?
Signals always have a current value and are synchronous; Observables are streams over time. They interoperate via `toSignal` and `toObservable`.

I. Practice Exercises

1. Create a `quantity` and `price` signal and a `computed` total including 18% GST.
2. Use `effect` to save a counter value into `localStorage` every time it changes.
3. Build a to-do counter showing the number of completed tasks with `computed`.